AI ASSISTANT CODING - ASSIGNMENT-19.3

Name: Valaboju Harshavardhanu

Roll No: 2303A52354 **Batch:** 41

Task 1: Python to C++ Conversion

Translate a simple Python class into C++ using AI assistance.

Prompt:

Convert a Python class Student with attributes name, age and a display() method into equivalent C++ code with constructor and proper access specifiers.

Code:

Python code:

```python
class Student:
    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age

    def display(self) -> None:
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")


if __name__ == "__main__":
    student = Student("Harsh", 20)
    student.display()
```

C++ code:

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string name;
    int age;

public:
    Student(const std::string& studentName, int studentAge)
        : name(studentName), age(studentAge) {}

    void display() const {
        std::cout << "Name: " << name << std::endl;
        std::cout << "Age: " << age << std::endl;
    }
};

int main() {
    Student student("Harsh", 20);
    student.display();
    return 0;
}
```

Output:

```
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab> .\student
Name: Harsh
Age: 20
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

Justification:

This task demonstrates understanding of object-oriented programming across languages. It highlights differences in syntax, data types, and memory handling between Python and C++.Using AI helps speed up code translation while maintaining logic consistency. It also improves knowledge of constructors and access specifiers in C++.

Task 2: Java to python Function conversion

Convert a Java method that checks whether a number is prime into

Python.

Prompt:

Convert Java method isPrime(int n) into Python function using Pythonic syntax.

Code:

Java code:

```java
public static boolean isPrime(int n) {
    if(n <= 1) return false;
    for(int i=2; i<=n/2; i++) {
        if(n % i == 0) return false;
    }
    return true;
}
```

Python code:

```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n//2 + 1):
        if n % i == 0:
            return False
    return True

print(is_prime(7))
print(is_prime(10))
```

Output:

```
True
False
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

Justification:

This task focuses on translating logic from a statically typed language to a dynamically typed one. It helps in understanding Pythonic syntax and simpler control structures. AI assists in reducing manual effort while preserving algorithm correctness. Testing ensures the logic works correctly across different inputs.

Task 3: Contact Form with Client-Side Validation

Translate a given pseudocode algorithm into Python using AI.

Prompt:

Convert bubble sort pseudocode into Python program.

Code:

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

data = [5, 2, 9, 1]
print(bubble_sort(data))
```

Output:

```
venv\Scripts\python.exe" "c:/Users/harsh/OneDrive/Desktop
[1, 2, 5, 9]
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

Justification:

This task converts human-readable logic into executable code. It strengthens problem-solving and algorithm implementation skills. AI helps in quickly transforming pseudocode into working programs. Validation ensures the implementation matches the intended logic.

Task 4: Dynamic List Management for a Product Page

Translate a SQL query into a Pandas DataFrame operation in

Prompt:

Convert SQL query SELECT name, salary FROM employees WHERE salary > 50000 into Pandas code.

Code:

```
-- SQLite
ELECT name, salary
FROM employees
WHERE salary > 50000;
```

Python pandas code:

```
import pandas as pd

data = {
    "name": ["A", "B", "C"],
    "salary": [40000, 60000, 70000]
}

df = pd.DataFrame(data)

result = df[df["salary"] > 50000][["name", "salary"]]
print(result)
```

Output:

```
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab> python ass.py
  name  salary
1    B   60000
2    C   70000
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

Justification:

This task bridges database querying and data analysis in Python. It shows how SQL operations can be performed using Pandas DataFrames. AI simplifies conversion of query logic into Python code. It enhances understanding of real-world data handling and filtering.

Task 5: Real Time Application: Algorithm Translation Across

Prompt:

Convert Python linear search algorithm into C language with proper memory handling.

Python Code:

```python
def search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

print(search([1,2,3,4], 3))
```

C Code:

```c
#include <stdio.h>

int search(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int arr[] = {1,2,3,4};
    int result = search(arr, 4, 3);
    printf("%d\n", result);
    return 0;
}
```

Output:

```
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab> .\ass
Element found at index 2
```

Justification:

This task demonstrates cross-language algorithm implementation skills. It highlights differences in syntax, memory management, and execution. AI helps in efficient translation while preserving core logic. It improves adaptability to work with multiple programming languages.